ENGR 102 Summer Bridge
Bridge Day 15 Student Workbook

Append, Concatenation, Slicing, and Strings as Sequences

Friday, July 24, 2026 • 50 points • tagged v5 successor

Name		**UIN**		**Section**	

Daily target and independence boundary

Process strings by index and build stored results with append, concatenation, and direct slice boundaries.

Allowed tools	Prior tools plus string indexing, in, direct slicing, list append, and list/string concatenation
Support supplied	The input/output contract and allowed sequence tools are supplied.
Student decides	Traversal state, collected-result state, slice boundaries, and validation order.
Scaffold level	Specification only; no planning prompts, variable inventory, or ordered algorithm.

Evidence and scoring

Evidence	**Points**	**Secure work shows**
Integrated trace	9	intermediate state, condition results, exact output, and meaning
Diagnosis and repair	7	actual, intended, cause, repair, and a discriminating test
Complete program and tests	34	coherent executable logic, required output, assumptions, and defended tests

Task 1. Integrated trace - 9 points

Trace index selection, append state, slicing, and concatenation in one sequence ledger.

```
labels = ["A", "B"]
text = "loop"

for index in range(1, len(text), 2):
    labels.append(text[index])

tail = text[1:3]
combined = labels + [tail]
print(labels)
print(tail)
print(combined)
```

Your task

1. Give the complete index/list/slice state and all exact output lines. Explain why the slice stop is not included.

Task 2. Diagnose and repair - 7 points

The intent is to create [1, 2, 3], but the program fails before print.

```
values = [1, 2]
values = values + 3
print(values)
```

Your task

1. Name the actual failure, explain the type mismatch, give two minimal repairs with different sequence operations, and name one empty-list test.

Task 3. Construct a complete program - 34 points

Program specification

- Read word as text. If its length is less than 4, print invalid and nothing else.
- Otherwise loop through every index and append characters at even indexes to even_characters.
- Count vowels using membership in the text aeiouAEIOU.
- Build edge_text by concatenating the first two-character slice and the last two-character slice.
- Print even_characters, vowel_count, and edge_text on separate lines.

Program workspace

Task 3 continued. Test and defend - included in 34 points

Continue code if needed. Required tests, expected results, and brief defense

Bridge Day 15 VIP Evidence Handoff

STUDENT COPY • Contract 07a04826c975 • Accessible v5 successor

Why engineers use this page

Apply filtering and the same normalization pipeline to strings before counting or comparing.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: string traversal, list traversal, list index read, list index write, string index read, slice expression, membership test, parallel list indexing, compound filter
Carried authoring tools: assignment, print call, arithmetic expression, input call, int conversion, float conversion, f string, comparison expression, boolean operator, if elif else, for loop, range call, append call, len call
Supplied-code exposure only: none
An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.23	Count Kept Code Characters	`18-23-work / 18-23-transfer / 18-23-cold`	arithmetic expression, for loop
18.24	Badge ID Match	`18-24-work / 18-24-transfer / 18-24-cold`	comparison expression

Readiness evidence

1. For Activity 18.23, name the characters that do not count and the condition that skips them.

2. For Activity 18.24, write the cleanup operations in the same order for both identifiers.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	
Observed Result	
Revision Reason	
Engineering Meaning	
Units Or Not Applicable	
Assumption	
Model Limit	
Next Test	
Help Trigger	
Minutes Used	

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Generated from the canonical VIP construct and evidence contract; workbook planning and executable notebook work remain distinct.